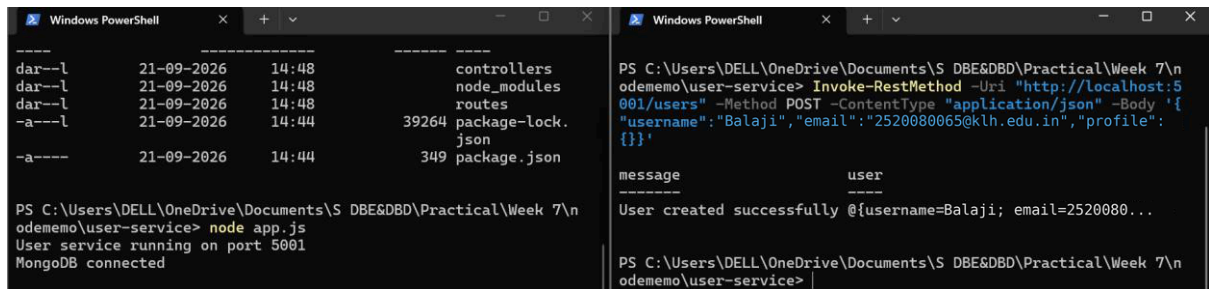


WEEK-7 PRACTICAL — Node.js User Service [Balaji, 2520080065]

Step – 1 Create a User — POST



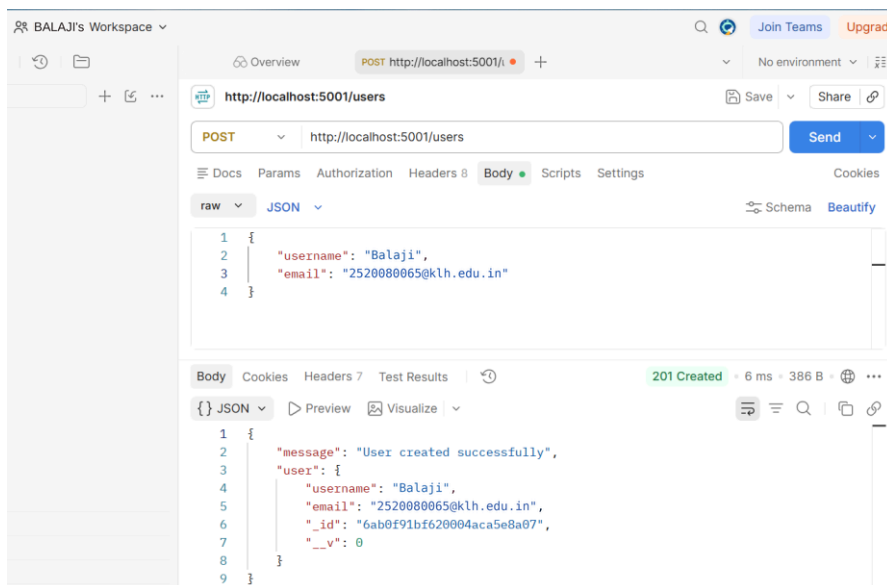
```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\nodememo\user-service> node app.js
User service running on port 5001
MongoDB connected
```

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\nodememo\user-service> Invoke-RestMethod -Uri "http://localhost:5001/users" -Method POST -ContentType "application/json" -Body '{"username":"Balaji","email":"2520080065@klh.edu.in","profile":{}}'
```

message user

User created successfully @{username=Balaji; email=2520080065@klh.edu.in; profile={}}

POST in mongosh:



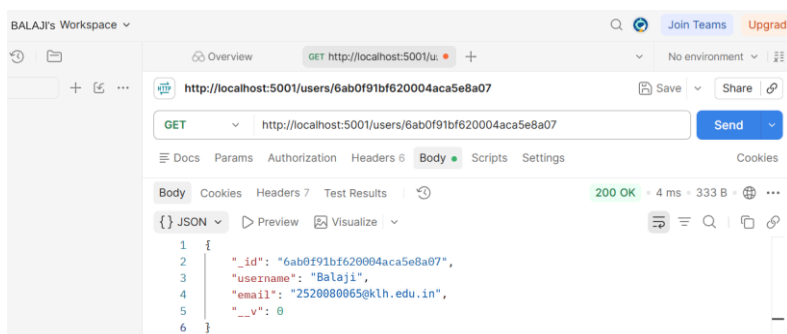
```
POST http://localhost:5001/users
```

```
{
  "username": "Balaji",
  "email": "2520080065@klh.edu.in"
}
```

201 Created • 6 ms • 386 B

```
{
  "message": "User created successfully",
  "user": {
    "username": "Balaji",
    "email": "2520080065@klh.edu.in",
    "_id": "6ab0f91bf620004aca5e8a07",
    "__v": 0
  }
}
```

Step 2: Get User — GET

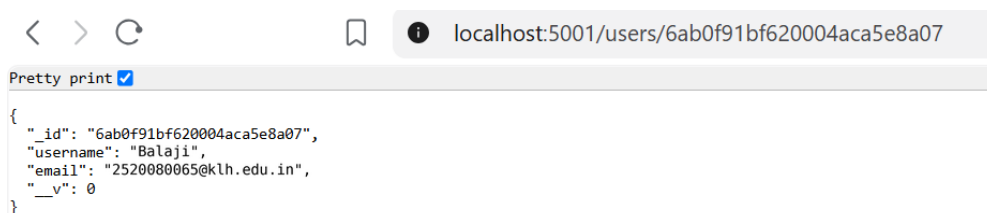


```
GET http://localhost:5001/users/6ab0f91bf620004aca5e8a07
```

200 OK • 4 ms • 333 B

```
{
  "_id": "6ab0f91bf620004aca5e8a07",
  "username": "Balaji",
  "email": "2520080065@klh.edu.in",
  "__v": 0
}
```

Output on browser by id:



```
localhost:5001/users/6ab0f91bf620004aca5e8a07
```

Pretty print

```
{
  "_id": "6ab0f91bf620004aca5e8a07",
  "username": "Balaji",
  "email": "2520080065@klh.edu.in",
  "__v": 0
}
```

Step 3: Create Activity — POST

```
PS C:\Users\DELL\OneDrive\Documents\S DBESDBD\Practical\Week 7\nodememo\user-service> Invoke-RestMethod -Uri "http://localhost:5001/activities" -Method POST -ContentType "application/json" -Body '{"user_id":"6ab0f91bf620004aca5e8a07","action":"SEARCH_PYTHON"}'

message          activity
-----
Activity created successfully @{user_id=6ab0f91bf620004aca5e...
```

In mongosh:

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5001/activities`
- Method:** `POST`
- Body (raw JSON):**

```
{
  "user_id": "6ab0f91bf620004aca5e8a07",
  "action": "SEARCH_JAVA"
}
```
- Response:** `201 Created` (9 ms, 439 B)
- Response Body (raw JSON):**

```
{
  "message": "Activity created successfully",
  "activity": {
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "SEARCH_JAVA",
    "id": "6ab0fb9ef620004aca5e8a08",
    "timestamp": "2026-09-21T09:40:46.119Z",
    "__v": 0
  }
}
```

Step 4: Add More Activities

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5001/activities/6ab0f91bf620004aca5e8a07`
- Method:** `POST`
- Body (raw JSON):**

```
[
  {
    "action": "Perform Week 7 Skill1"
  },
  {
    "action": "Perform Week 7 Practical"
  },
  {
    "action": "Complete MongoDB Task"
  },
  {
    "action": "Test APIs in Postman"
  }
]
```
- Response:** `201 Created` (9 ms, 439 B)
- Response Body (raw JSON):**

```
[
  {
    "id": "6ab10015f620004aca5e8a0c",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "SEARCH_PYTHON",
    "timestamp": "2026-09-21T09:59:49.278Z",
    "__v": 0
  },
  {
    "id": "6ab0fe44f620004aca5e8a09",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "Perform Week 7 Practical",
    "timestamp": "2026-09-21T09:52:04.478Z",
    "v": 0
  }
]
```

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\nodememo\user-service> Invoke-RestMethod -Uri "http://localhost:5001/activities/6ab0f91bf620004aca5e8a07" -Method GET

_id      : 6ab10015f620004aca5e8a0c
user_id  : 6ab0f91bf620004aca5e8a07
action   : SEARCH_PYTHON
timestamp : 2026-09-21T09:59:49.270Z
__v      : 0

_id      : 6ab0fe44f620004aca5e8a09
user_id  : 6ab0f91bf620004aca5e8a07
action   : Perform Week 7 Practical
timestamp : 2026-09-21T09:52:04.478Z
__v      : 0

_id      : 6ab0fb9ef620004aca5e8a08
user_id  : 6ab0f91bf620004aca5e8a07
action   : SEARCH_JAVA
timestamp : 2026-09-21T09:40:46.119Z
__v      : 0
```

Output on browser:

localhost:5001/activities/6ab0f91bf620004aca5e8a07

Pretty print ☒

```
[
  {
    "_id": "6ab10015f620004aca5e8a0c",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "SEARCH_PYTHON",
    "timestamp": "2026-09-21T09:59:49.270Z",
    "__v": 0
  },
  {
    "_id": "6ab0fe44f620004aca5e8a09",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "Perform Week 7 Practical",
    "timestamp": "2026-09-21T09:52:04.478Z",
    "__v": 0
  },
  {
    "_id": "6ab0fb9ef620004aca5e8a08",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "SEARCH_JAVA",
    "timestamp": "2026-09-21T09:40:46.119Z",
    "__v": 0
  }
]
```

MongoDB connection:

MongoDB Compass - 127.0.0.1:27017/digital_library

Connections Edit View Help

Compass

127.0.0.1:27017 > digital_library

Collection name |# Properties |# Storage size |# Data size |# Documents |# Avg. document size |#

Collection name	#	Properties	#	Storage size	#	Data size	#	Documents	#	Avg. document size	#
activities	-			32.77 kB		300.00 B		3		100.00 B	
users	-			36.86 kB		339.00 B		4		84.00 B	

MongoDB Compass - 127.0.0.1:27017/digital_library.activities

Connections Edit View Collection Help

Compass

127.0.0.1:27017 > digital_library > activities

Documents | Aggregations | Schema | Indexes | Validation

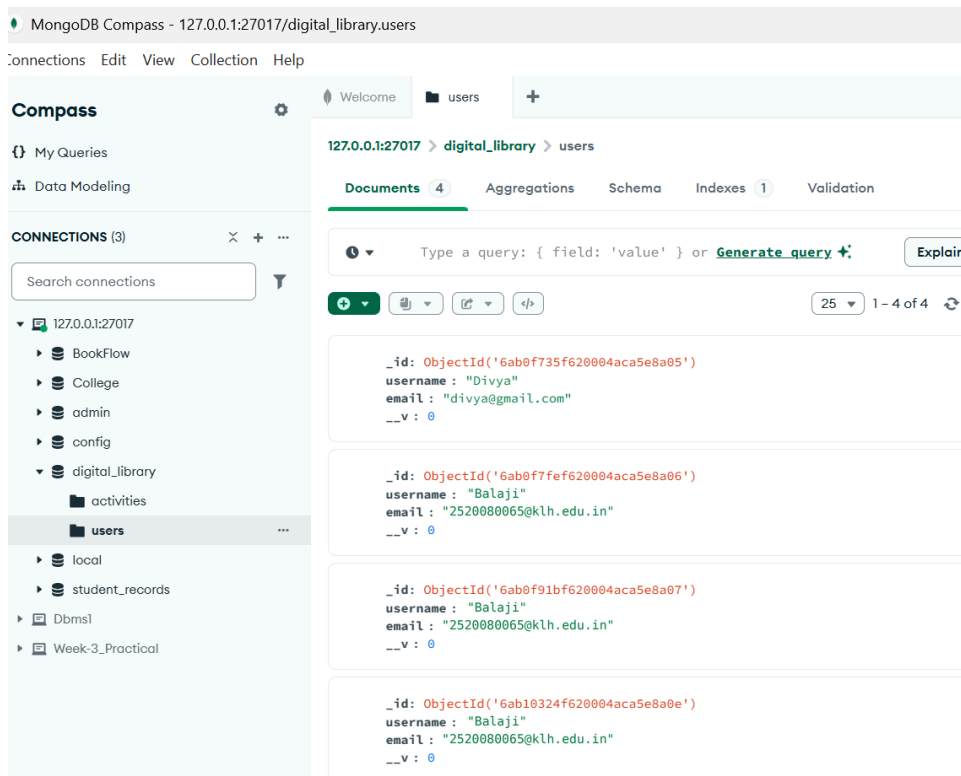
Type a query: { field: 'value' } or Generate query

Find 1-3 of 3

```
{
  "_id": ObjectId("6ab0fb9ef620004aca5e8a08"),
  "user_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "action": "SEARCH_JAVA",
  "timestamp": ISODate("2026-09-21T09:40:46.119+00:00"),
  "__v": 0
}
```

```
{
  "_id": ObjectId("6ab0fe44f620004aca5e8a09"),
  "user_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "action": "Perform Week 7 Practical",
  "timestamp": ISODate("2026-09-21T09:52:04.478+00:00"),
  "__v": 0
}
```

```
{
  "_id": ObjectId("6ab10015f620004aca5e8a0c"),
  "user_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "action": "SEARCH_PYTHON",
  "timestamp": ISODate("2026-09-21T09:59:49.270+00:00"),
  "__v": 0
}
```



Folder Structure:

Week 7

```

|__nodememo
  |__user-service
    |
    |__ controllers
      |__ activityController.js
      |__ userController.js
    |
    |__ routes
      |__ activityRoutes.js
      |__ userRoutes.js
    |
    |__ app.js
    |__ db.js
    |__ package.json
    |__ .env

```

db.js

```
const mongoose = require("mongoose");

const MONGO_URI =
  process.env.MONGO_URI ||
  "mongodb://127.0.0.1:27017/digital_library";

mongoose.connect(MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error("MongoDB connection error:", err));

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true
  },
  profile: {
    type: mongoose.Schema.Types.Mixed,
    default: {}
  }
});

const activitySchema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User"
  },
  action: {
    type: String,
    required: true
  },
  timestamp: {
    type: Date,
    default: Date.now
  }
});

const User = mongoose.model("User", userSchema);
const Activity = mongoose.model("Activity", activitySchema);

module.exports = { User, Activity };
```

controllers/userController.js

```
const { User } = require("../db");

const createUser = async (req, res) => {
  try {
    const { username, email, profile } = req.body;

    const user = await User.create({
      username,
      email,
      profile
    });

    res.status(201).json({
      message: "User created successfully",
      user
    });
  } catch (error) {
    res.status(500).json({
      message: "Error creating user",
      error: error.message
    });
  }
};

const getUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);

    if (!user)
      return res.status(404).json({
        message: "User not found"
      });

    res.json(user);
  } catch (error) {
    res.status(500).json({
      message: "Error getting user",
      error: error.message
    });
  }
};

module.exports = { createUser, getUser };
```

controllers/activityController.js

```
const mongoose = require("mongoose");
const { Activity } = require("../db");

const createActivity = async (req, res) => {
  try {
    const { user_id, action } = req.body;

    if (!mongoose.Types.ObjectId.isValid(user_id))
      return res.status(400).json({
        message: "Invalid user_id"
      });

    const activity = await Activity.create({
      user_id,
      action
    });

    res.status(201).json({
      message: "Activity created successfully",
      activity
    });
  } catch (error) {
    res.status(500).json({
      message: "Error creating activity",
      error: error.message
    });
  }
};

const getActivities = async (req, res) => {
  try {
    const { user_id } = req.params;

    if (!mongoose.Types.ObjectId.isValid(user_id))
      return res.status(400).json({
        message: "Invalid user_id"
      });

    const activities = await Activity
      .find({ user_id })
      .sort({ timestamp: -1 });

    res.json(activities);
  } catch (error) {
    res.status(500).json({
      message: "Error getting activities",
    });
  }
};
```

```

        error: error.message
    });
}
};

module.exports = { createActivity, getActivities };

```

routes/userRoutes.js

```

const express = require("express");

const router = express.Router();

const {
    createUser,
    getUser
} = require("../controllers/userController");

router.post("/users", createUser);
router.get("/users/:id", getUser);

module.exports = router;

```

routes/activityRoutes.js

```

const express = require("express");

const router = express.Router();

const {
    createActivity,
    getActivities
} = require("../controllers/activityController");

router.post("/activities", createActivity);
router.get("/activities/:user_id", getActivities);

module.exports = router;

```

app.js

```

const express = require("express");

const app = express();

require("../db");

app.use(express.json());

```

```
const activityRoutes = require("./routes/activityRoutes");
const userRoutes = require("./routes/userRoutes");

app.use("/", activityRoutes);
app.use("/", userRoutes);

app.listen(5001, () => {
  console.log("User service running on port 5001");
});
```